EN: 6.

Quick Work
Page No.:
Date:
M T W T F S S

ERROR CORRECTION AND DETECTION USING HAMMING CODE

AIM:

To Simulate hamming code data to detect and correct it.

PROCEDURE: ALGORITH.

* Start The program.

* Read The number of bits / bits.

* Using The hamming Code concept find. redundent bits assuming even parity.

* Add The redundant bits in The data at recording places.

* Send. The data to recevis side.
* Find The redundant bits again.
* Take is complement.
* Stop The program.

Code:

```
def Calculate_parity_bits (dat):
    M = len (data)
    r=0
    while (2**r) < (m + r + 1):
        r+=1
    return r
```

```
def generate_hamming_code(data):
    m = len(data)
    r = calculate_parity_bits(data)
    n = m + r
    hamming = ['0'] * (n+1)
    j = 0
    for i in range(1, n+1):
        if range(1, n+1):
            hamming[i] = data[j]
            j += 1
    for r in range(r):
        position = 2 ** p
        parity = 0
        for r in range(n+1):
            if j position:
                parity n = n + (hamming)[i]
        hamming[position] = str(parity)
    return "".join(hamming[1:])

def defect_and_correct(recived):
    n = len(recived)
    r = 0
    while (2**r) < (n+1):
        r += 1
    hamming = ['0'] + list(recived)
    error_position = 0
    if error_postion != 0:
```

Output:

```
Enter the binary data: 1011
Original Data : 1011
Hamming Code : 0110011
Enter the recived Hamming Code : 0110111
Enter position : 5
Error deleted at posino 5
Error corrected Successfully!
Recived Code : 0110111
Corrected Code : 0110011
```

```
else:
    hamming[error_position] = '0'
    return ''.join(hamming[1:])
print("Hamming Code")
data = input("Enter the binary data:")
if not all(bit in '01' for bit in data):
    print("please enter only 0 and 1")
else:
    hamming_code = generate_hamming_code(data)
    print("Hamming Code", hamming_code)
if len(recived) != len(hamming_code) or
    not all(bit in '01' bit in recived):
else:
    corrected_code = data + and + correct(recive)
```

Result:

The Hamming Code program was Succesfully Implemented in Python.